

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 8

Student Name: G. Vijay Anudeep

Roll No.: A24126552081

Date: 16-08-2026

1. TITLE

Implementation of HTTP Methods using JSON Server

2. AIM

To implement and demonstrate the **GET, POST, PUT and DELETE HTTP methods** using JSON Server for managing data in a RESTful web application.

3. OBJECTIVES

1. To understand the concept of a REST API and JSON Server.
2. To create a simple JSON-based REST API.
3. To perform **GET, POST, PUT and DELETE** operations.
4. To understand how client applications communicate with a server using HTTP methods.
5. To test API endpoints using a suitable tool such as **Postman** or a web browser.

4. THEORY

JSON Server is a lightweight tool that allows developers to create a REST API from a JSON file without developing a complete backend server.

It uses HTTP methods to perform operations on resources:

HTTP Method	Purpose	Example
GET	Retrieve data	Get all students
POST	Add new data	Add a student
PUT	Update existing data	Update student details
DELETE	Remove data	Delete a student

For this experiment, a `db.json` file is used as the database.

Example resource:

```
{
  "students": [
    {
      "id": 1,
      "name": "Ravi",
      "branch": "CSE",
      "marks": 85
    }
  ]
}
```

When JSON Server is started, it automatically provides REST endpoints such as:

```
GET      /students
GET      /students/1
POST     /students
PUT      /students/1
DELETE   /students/1
```

Thus, JSON Server helps students understand RESTful API operations before working with a full backend framework.

`index.html` :

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">

  <title>Student Management System</title>

  <style>
```

```
body {  
    font-family: Arial, sans-serif;  
    margin: 40px;  
    background-color: #f4f4f4;  
}
```

```
h1 {  
    text-align: center;  
}
```

```
.form-container {  
    background: white;  
    padding: 20px;  
    margin-bottom: 25px;  
    border-radius: 8px;  
}
```

```
input {  
    padding: 10px;  
    margin: 5px;  
}
```

```
button {  
    padding: 9px 14px;  
    margin: 5px;  
    cursor: pointer;  
}
```

```
table {  
    width: 100%;  
    background: white;  
    border-collapse: collapse;  
}
```

```
th,  
td {
```

```

padding: 12px;
border: 1px solid #ccc;
text-align: center;
}

th {
background-color: #333;
color: white;
}
</style>
</head>

<body>
<h1>Student Management System</h1>

<div class="form-container">
<h2>Add Student</h2>
<input type="text" id="name" placeholder="Student Name">
<input type="text" id="branch" placeholder="Branch">
<input type="number" id="marks" placeholder="Marks">
<button onclick="addStudent()">Add Student</button>
<button onclick="getStudents()">Refresh</button>
</div>

<div class="form-container">
<h2>Student Records</h2>
<table>
<thead>
<tr>
<th>ID</th>
<th>Name</th>
<th>Branch</th>
<th>Marks</th>
<th>Actions</th>
</tr>
</thead>

<tbody id="studentTable">
</tbody>

</table>

```

```
</div>
<script src="script.js"></script>
</body>
</html>
```

script.js

```
const API_URL = "http://localhost:3000/students";

// GET: Display all students
async function getStudents() {

  try {

    const response = await fetch(API_URL);
    const students = await response.json();
    const table = document.getElementById("studentTable");
    table.innerHTML = "";

    students.forEach(student => {
      const row = document.createElement("tr");
      row.innerHTML = `
        <td>${student.id}</td>
        <td>${student.name}</td>
        <td>${student.branch}</td>
        <td>${student.marks}</td>

        <td>
          <button onclick="updateStudent('${student.id}')">
            Update
          </button>

          <button onclick="deleteStudent('${student.id}')">
            Delete
          </button>
        </td>
      `;
      table.appendChild(row);
    });
  }
}
```

```

    } catch (error) {
        console.error("Error:", error);
        alert("Unable to fetch student records.");
    }
}

// POST: Add a new student
async function addStudent() {

    const name = document.getElementById("name").value;
    const branch = document.getElementById("branch").value;
    const marks = document.getElementById("marks").value;

    if (name === "" || branch === "" || marks === "") {
        alert("Please enter all student details.");
        return;
    }

    const student = {
        name: name,
        branch: branch,
        marks: Number(marks)
    };

    try {
        const response = await fetch(API_URL, {
            method: "POST",
            headers: {
                "Content-Type": "application/json"
            },
            body: JSON.stringify(student)
        });

        if (response.ok) {
            alert("Student added successfully.");

            document.getElementById("name").value = "";
            document.getElementById("branch").value = "";
            document.getElementById("marks").value = "";
        }
    }
}

```

```

        getStudents();
    }
} catch (error) {
    console.error("Error:", error);
    alert("Unable to add student.");
}
}

```

// PUT: Update student

```

async function updateStudent(id) {

    try {
        // Get current student details
        const response = await fetch(`${API_URL}/${id}`);
        const student = await response.json();

        // Ask for updated values
        const name = prompt("Enter student name:", student.name);

        if (name === null) {
            return;
        }

        const branch = prompt("Enter branch:", student.branch);

        if (branch === null) {
            return;
        }

        const marks = prompt("Enter marks:", student.marks);

        if (marks === null) {
            return;
        }
    }
}

```

```

const updatedStudent = {
  name: name,
  branch: branch,
  marks: Number(marks)
};

// Update using PUT
const updateResponse = await fetch(`${API_URL}/${id}`, {
  method: "PUT",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(updatedStudent)
});

if (updateResponse.ok) {
  alert("Student updated successfully.");
  getStudents();
}

} catch (error) {
  console.error("Error:", error);
  alert("Unable to update student.");
}
}

// DELETE: Delete student
async function deleteStudent(id) {

  const confirmDelete = confirm(
    "Are you sure you want to delete this student?"
  );

  if (!confirmDelete) {
    return;
  }

```

```

try {

    const response = await fetch(`${API_URL}/${id}`, {
        method: "DELETE"
    });

    if (response.ok) {
        alert("Student deleted successfully.");
        getStudents();
    }

} catch (error) {
    console.error("Error:", error);
    alert("Unable to delete student.");
}

}

// Load students when page opens
getStudents();

```

OUTPUT:

The output shows four screenshots of a web application titled "Student Management System". Each screenshot displays a form to "Add Student" and a table of "Student Records".

Screenshot 1 (Top Left): Shows the initial state with 3 students in the table.

ID	Name	Branch	Marks	Actions
1	Ravi	CSE	85	Update Delete
2	Priya	AIML	91	Update Delete
3	Kiran	CSE	78	Update Delete

Output 1: GET request displays all student records.

Screenshot 2 (Top Right): Shows the state after adding a new student (Anil).

ID	Name	Branch	Marks	Actions
1	Ravi	CSE	85	Update Delete
2	Priya	AIML	91	Update Delete
3	Kiran	CSE	78	Update Delete
4	Anil	CSE	88	Update Delete

Output 2: POST request successfully adds a new student record.

Screenshot 3 (Bottom Left): Shows the state after updating Ravi's details.

ID	Name	Branch	Marks	Actions
1	Ravi Kumar	CSE	90	Update Delete
2	Priya	AIML	91	Update Delete
3	Kiran	CSE	78	Update Delete
4	Anil	CSE	88	Update Delete

Output 3: PUT request successfully updates Ravi's details.

Screenshot 4 (Bottom Right): Shows the state after deleting student ID 3.

ID	Name	Branch	Marks	Actions
1	Ravi Kumar	CSE	90	Update Delete
2	Priya	AIML	91	Update Delete
4	Anil	CSE	88	Update Delete

Output 4: DELETE request successfully removes student ID 3.

5. ALGORITHM/PROCEDURE

Step 1

Install Node.js and npm on the system.

Step 2

Create a new project folder named:

`JSONServerDemo`

Step 3

Open the terminal in the project folder and initialize the project:

`npm init -y`

Step 4

Install JSON Server: `npm install json-server`

Step 5

Create a file named: `db.json`

Step 6

Add sample student data to `db.json`.

Step 7

Start JSON Server using: `npm run json-server --watch db.json --port 3000`

Step 8

Test the **GET** operation to retrieve all students.

Step 9

Test the **POST** operation to add a new student.

Step 10

Test the **PUT** operation to modify an existing student's details.

Step 11

Test the **DELETE** operation to remove a student.

Step 12

Verify the changes in `db.json` and record the outputs.

6. PROGRAM / CODE

1. `db.json`

Create the following file:

```
{
  "students": [
    {
      "id": "1",
      "name": "Ravi",
      "branch": "CSE",
      "marks": 85
    },
    {
      "id": "2",
      "name": "Priya",
      "branch": "AIML",
      "marks": 91
    },
    {
      "id": "3",
      "name": "Kiran",
      "branch": "CSE",
      "marks": 78
    }
  ]
}
```

2. Start JSON Server

```
npx json-server --watch db.json --port 3000
```

```
PS C:\Users\Admin\OneDrive\Desktop\front-end classes> npx json-server --watch db.json --port 3000
--watch/-w can be omitted, JSON Server 1+ watches for file changes by default
JSON Server started on PORT :3000
Press CTRL-C to stop
Watching db.json...
```

```
(~ ~ ~)
```

Index:

<http://localhost:3000/>

Static files:

Serving ./public directory if it exists

Endpoints:

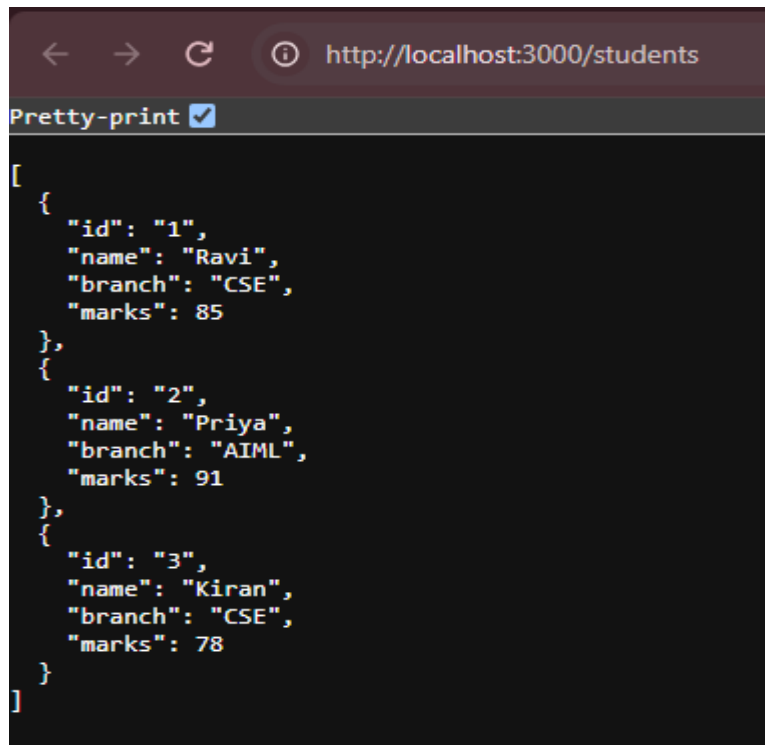
<http://localhost:3000/students>

HTTP Operations

A. GET Method

Retrieve all students

Method: GET

A screenshot of a web browser window. The address bar shows the URL 'http://localhost:3000/students'. Below the address bar, there is a 'Pretty-print' checkbox which is checked. The main content area displays a JSON array of three student objects. The first object has id '1', name 'Ravi', branch 'CSE', and marks 85. The second object has id '2', name 'Priya', branch 'AIML', and marks 91. The third object has id '3', name 'Kiran', branch 'CSE', and marks 78.

```
[
  {
    "id": "1",
    "name": "Ravi",
    "branch": "CSE",
    "marks": 85
  },
  {
    "id": "2",
    "name": "Priya",
    "branch": "AIML",
    "marks": 91
  },
  {
    "id": "3",
    "name": "Kiran",
    "branch": "CSE",
    "marks": 78
  }
]
```

B. POST Method

Add a new student

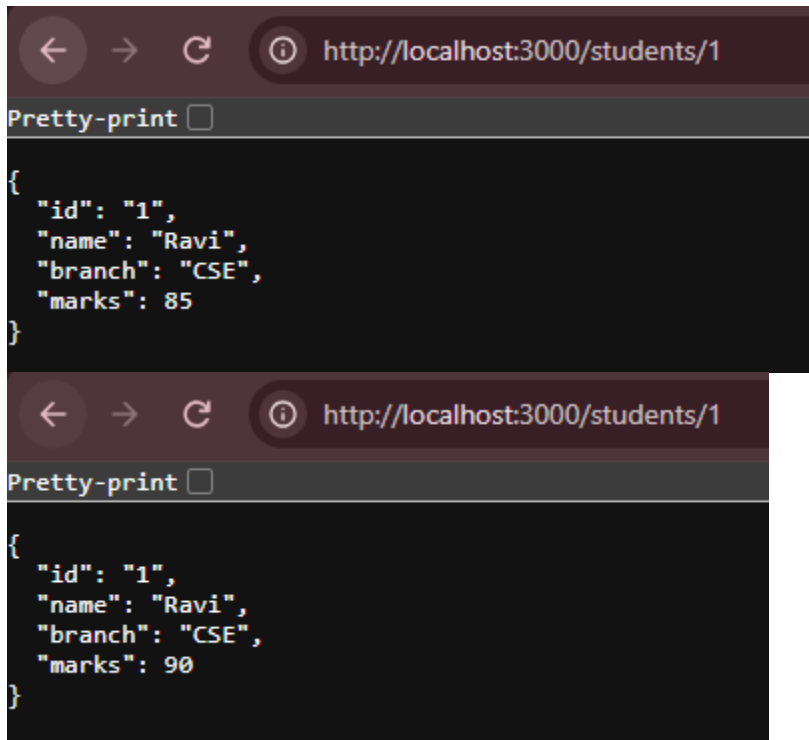
Method: POST

```
{
  "name": "Anil",
  "branch": "CSE",
  "marks": 88
}
```

C. PUT Method

Update student details

Method: PUT



The details of student with ID **1** will be updated.

D. DELETE Method

Delete a student

Method: DELETE

<http://localhost:3000/students/3> - The student with ID **3** will be removed.

7. TEST CASES AND EXECUTION DATA

Students should record at least the following test cases:

Test Case	HTTP Method	URL	Input / Operation	Expected Result
TC01	GET	/students	Retrieve all students	All student records are displayed
TC02	GET	/students/1	Retrieve student ID 1	Student ID 1 details are displayed
TC03	POST	/students	Add Anil, CSE, 88	New student record is created
TC04	PUT	/students/1	Change Ravi's marks to 90	Student ID 1 is updated
TC05	DELETE	/students/3	Delete student ID 3	Student ID 3 is deleted

Execution Data

Operation	Input	Actual Output	Status
GET all students	/students	3 student records displayed	Pass
GET student	/students/1	Ravi's details displayed	Pass
POST student	Anil, CSE, 88	New record added	Pass
PUT student	ID 1, marks = 90	Record updated	Pass
DELETE student	ID 3	Record deleted	Pass